

PPG 센서와 딥러닝을 활용한

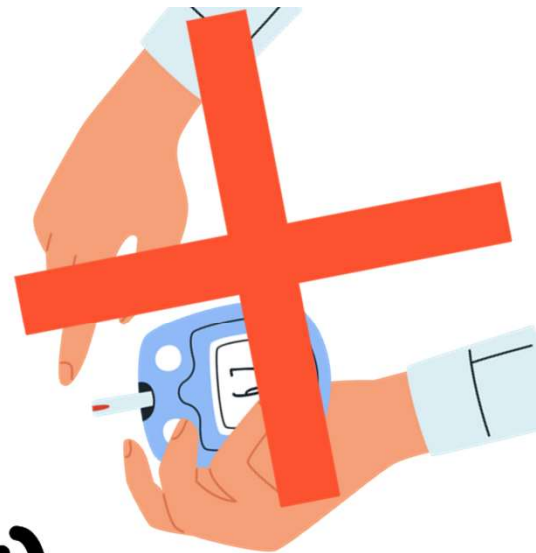
즐거로운 당뇨 생활

당뇨 관리를 위한 비침습 혈당 관리 서비스

팀명 | Cellu

발표자 | 박승주 (팀장)

팀원 | 김슬기, 박성현, 이규해, 장서우

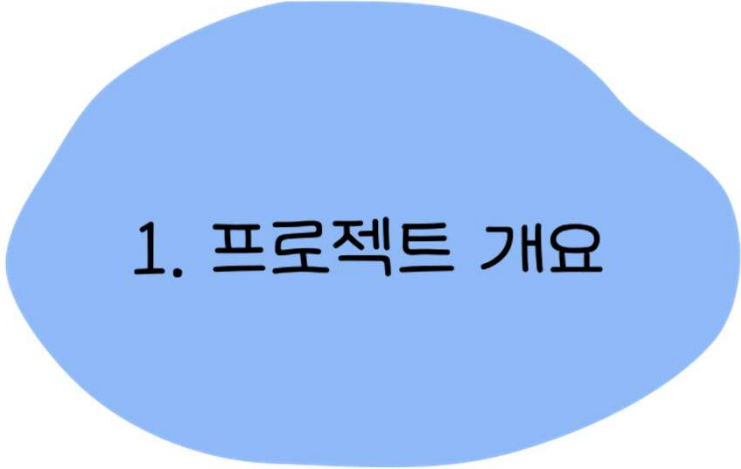


목차

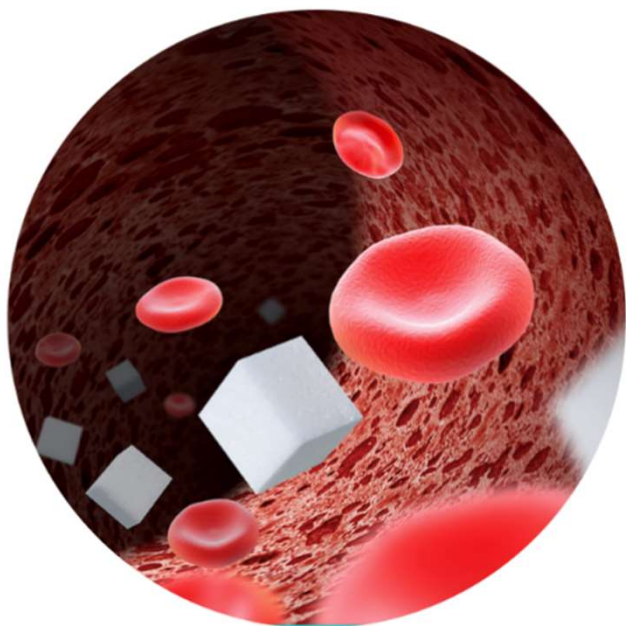
1. 프로젝트 개요

2. 개발 내용

3. 기대효과 및 활용 방안



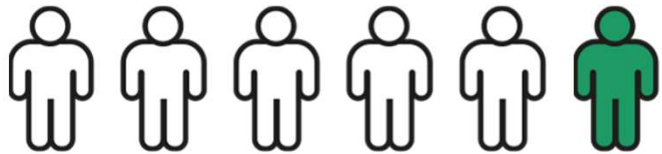
1. 프로젝트 개요



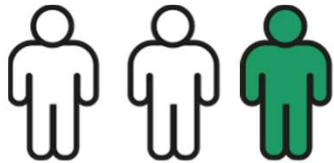
당뇨 ?

췌장의 인슐린 분비 이상
인슐린 저항성으로 인한 작동이상으로
당이 혈액 속에 과다하게 남아
온몸에 여러 문제를 일으키게 되는
만성 질환

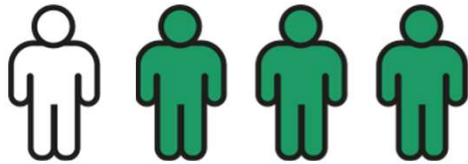
혈당 관리의 실태



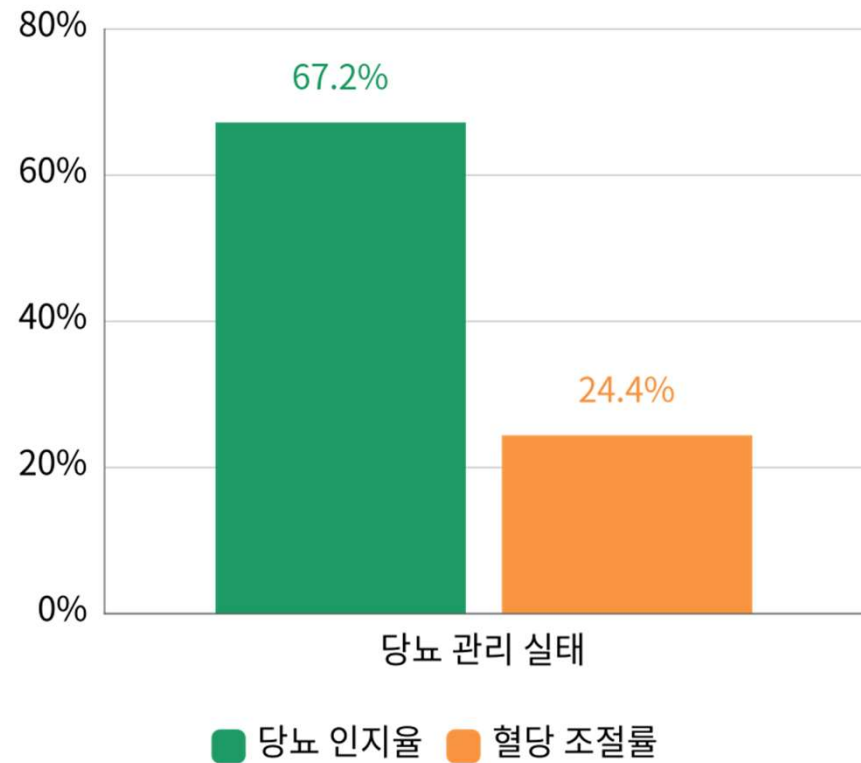
국민(만 30세 이상) 6명 중 1명이 당뇨병



당뇨 환자 3명 중 1명은 당뇨 인지 X



당뇨 환자 4명 중 3명은 혈당 조절 X



출처 : 국민건강통계 질병관리청

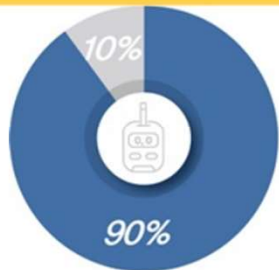
당뇨병이 만성화되면 전신으로 합병증이 발생합니다



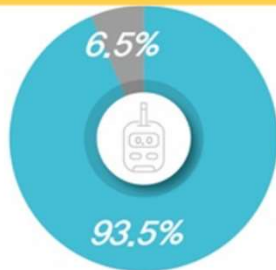
혈당 관리 어려움의 주요 요인



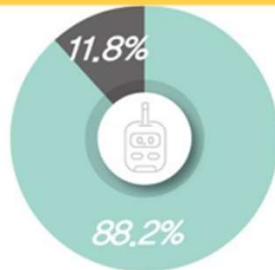
높은 유지비용



■ 사용
■ 비사용



■ 부담이 된다.
■ 부담이 되지 않는다.



■ 요양비 비율이 확대되어야 한다.
■ 만족한다.

자료 출처 : 한국 1형 당뇨병 환우회

출처 : 소비자가 만드는 신문



낮은 정확도

병원에 입원해 정밀검사를 받은 결과, 강 씨의 혈당치는 134mg/dL로 판명됐다.

연소현다츠전기 배운 고 언어 누치부는 '화자'

측정의 어려움

김재현 교수는 "연속혈당측정기가 있는 방법이 없는 것"이라
비유해도 마찬가지다. 기기값은 지원해주면서 운전하는 방법은 알려주지 않는 것과 마찬가지"라고

논문 검토

의료기술재평가보고서 2023

자율신경계이상검사
-심박변이도검사-심탄도를 이용한 연속적인 심박수 모니터링 및 당뇨 예측
가능성 연구(파일럿연구)최상기¹, 이거룡^{2*}¹선문대학교 통합의학과 박사수료, ²선문대학교 통합의학과 교수

심박변이도와 대사증후군의 연관성

조용진, 송동원

순천향대학교 의과대학 천안병원 가정의학과

심탄도와 인공지능을 이용한 혈당수치 예측모델 연구

최상기¹, 박철구^{2*}¹소프트웨어융합연구소 연구소장, ²소프트웨어융합연구소 대표



“ PPG 센서를 활용한 딥러닝 기반 비침습 혈당 측정 ”

PPG + 딥러닝

기존 제품

낮은 정확도

교체형으로 높은 유지비용

주사나 채혈의 통증과 의료기기의 불편함

혈당 관리의 어려움

측정 기기의 불편한 휴대성



95% 이상의 높은 정확도

비교체형으로 낮은 유지 비용

비침습으로 편리하고 쉽게 측정 가능

전용앱 제공으로 직관적인 혈당 관리

기기의 편리한 휴대성





2. 개발 내용

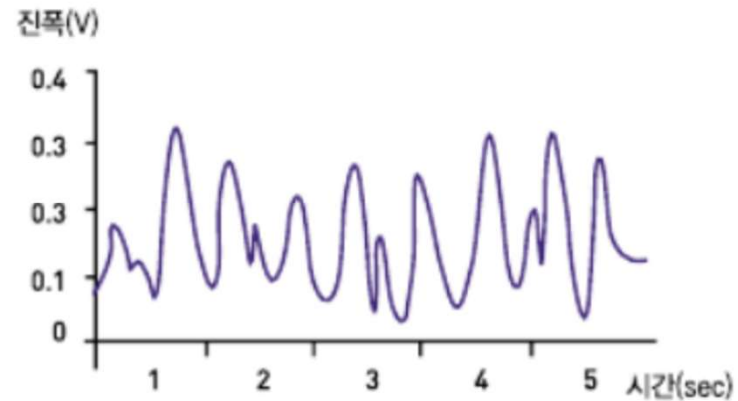
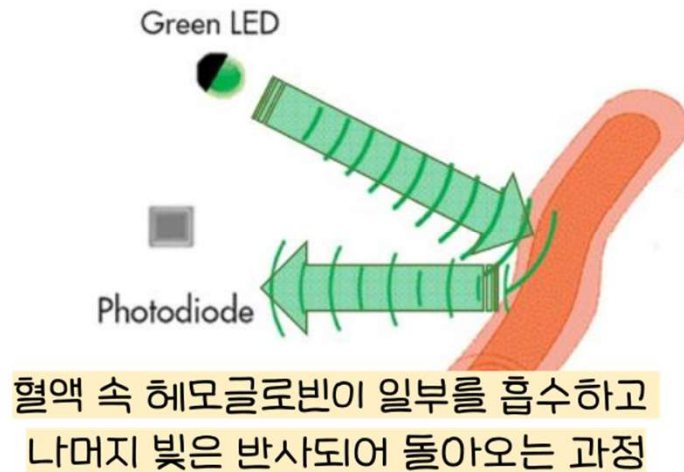
PPG ? Photo Plethysmo Graphy (광 혈류 측정)



Green Led 흡수율이 높고 외란광의 영향이 적음

photodiode 빛을 전류로 바꾸는 반도체 소자

혈관의 흐름 변화에 따라 빛의 흡수량이 변화 측정 -> 파형이 생김



출처 : 실험키트 블로그

데이터 구성 요소

PPG 센서 데이터

HR (심박도)
HRV (심박변이도)
SDNN (표준 편차)
RMSSD (제공근 평균 제공 차이)
PNN50 (NN 간격의 50% 이상 차이 비율)
VLF (가장 낮은 주파수)
LF (낮은 주파수)
HF (높은 주파수)
Frequency Ratio (주파수 비율)

+

사용자 데이터

Gender (성별)
Age (나이)

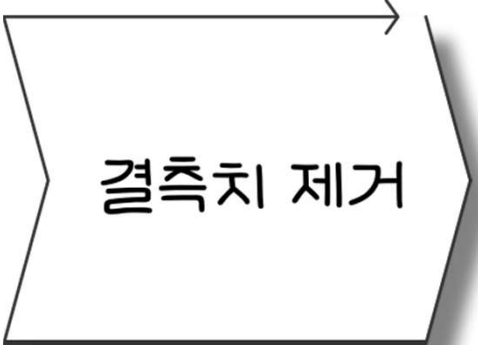
=

총 11개

데이터 전처리 흐름도



데이터 전처리

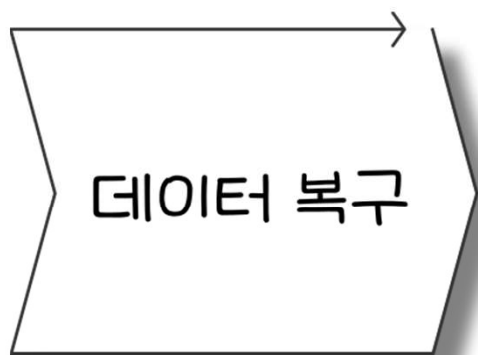


결측치 제거

$HR \leq 0$ 경우 전부 제거

심박도

데이터 전처리



결측치 해당 열 평균화 작업

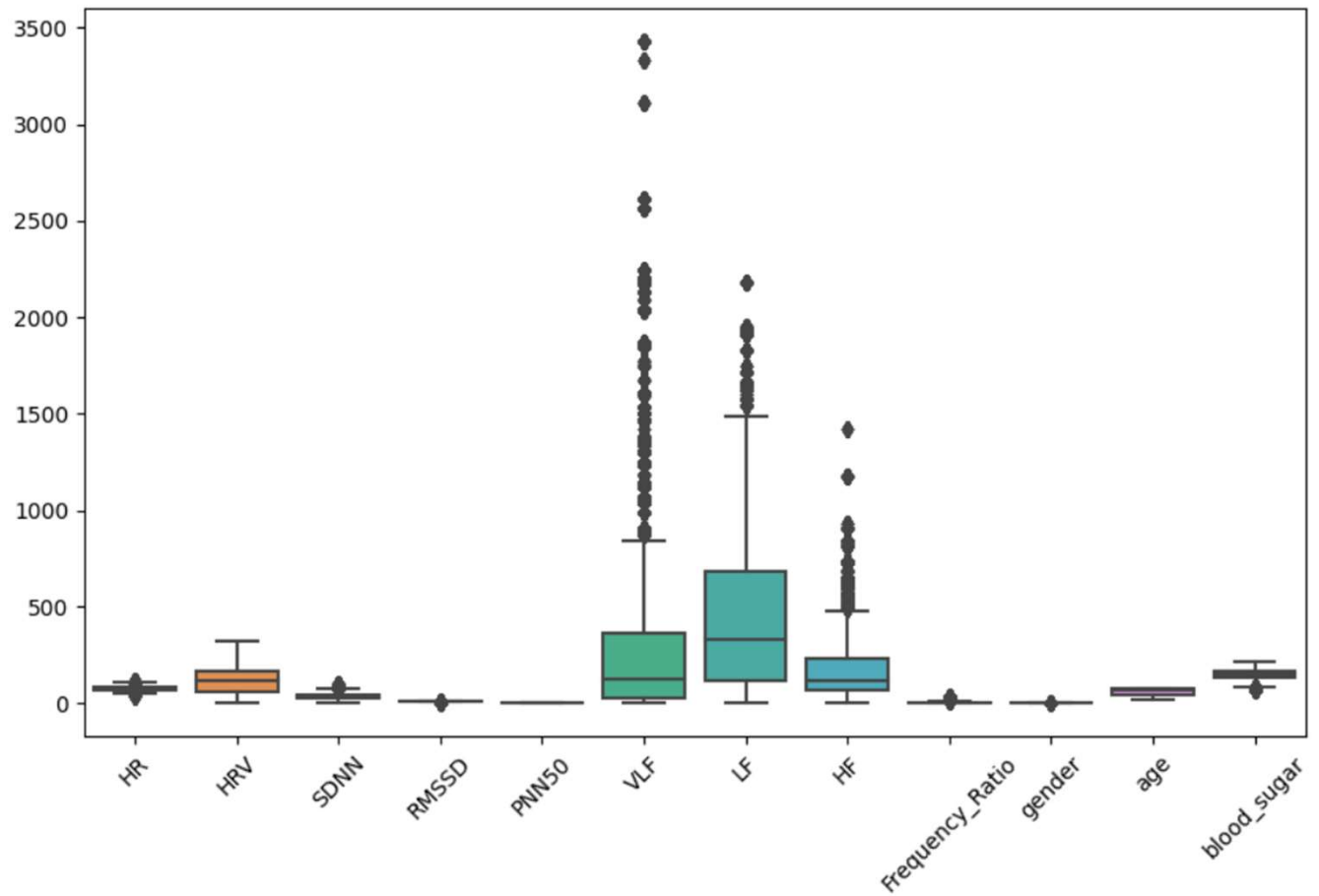
삭제나 다른 값으로 대체하지 않고
평균값으로 채워 유용한 정보 손실 방지

데이터 전처리

이상치 확인

박스플롯 활용 결과

VLF, LF, HF(주파수)
이상치 발견



데이터 전처리

Scaler 사용



Robust Scaler

Train Results:

MSE: 0.5147, MAE: 0.2869, RMSE: 0.7174, R^2 : 0.9984

Validation Results:

MSE: 0.7148, MAE: 0.3017, RMSE: 0.8454, R^2 : 0.9977

Standard Scaler

Final Train Results:

RMSE: 1.3257, MSE: 1.7574, MAE: 1.1315, R^2 : 0.9944

Final Validation Results:

RMSE: 1.3577, MSE: 1.8434, MAE: 1.1379, R^2 : 0.9942

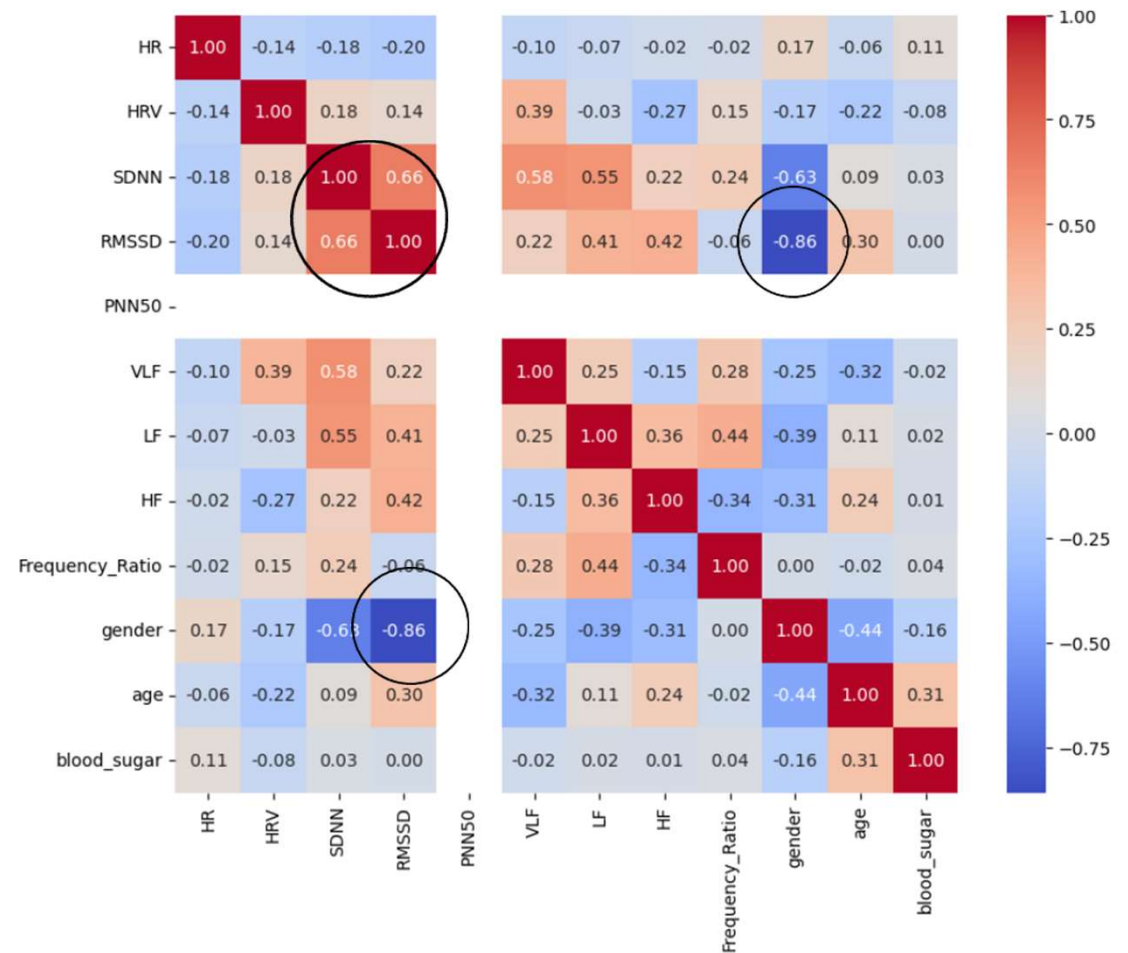
데이터 분석(EDA)

데이터셋의 변수 간
피어슨 상관관계를 분석 결과

RMSSD와 SDNN 간에는
강한 양의 상관관계

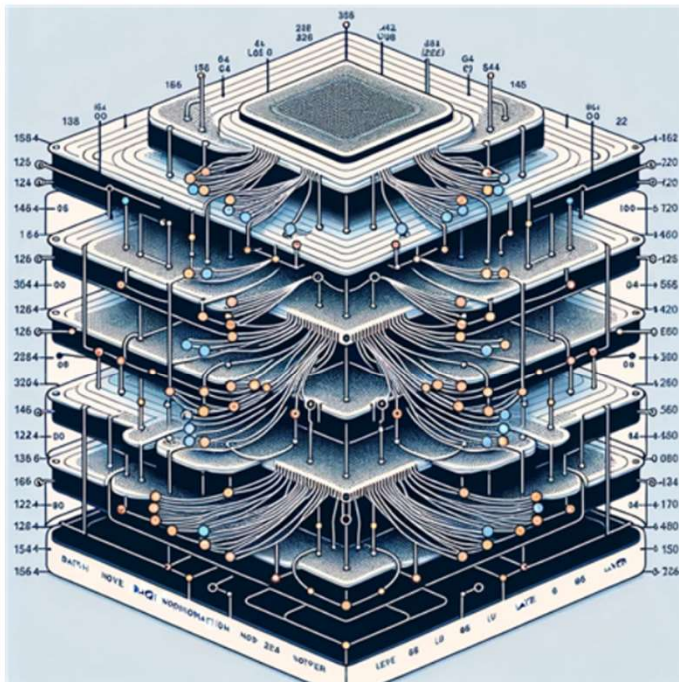
성별(gender)과 RMSSD는
매우 강한 음의 상관관계

상관관계가 있는 특성을 매개변수로 활용



모델 선택

DNN(Deep Neural Network) 기반의 MLP(Multi-Layer Perceptron) 모델 채택



지도 학습 및 분류 문제에 대한 우수한 성능 고려

데이터 분석 결과 선형적 특성의 데이터지만

ReLU 함수, Adam 컴파일을 통하여

비선형성 증가

비선형성이 증가하게 되면

모델 특징의 식별성 증가로 이어짐

모델 평가 및 검증

1. MAE, MSE, RMSE, 결정계수(R^2) 값 확인

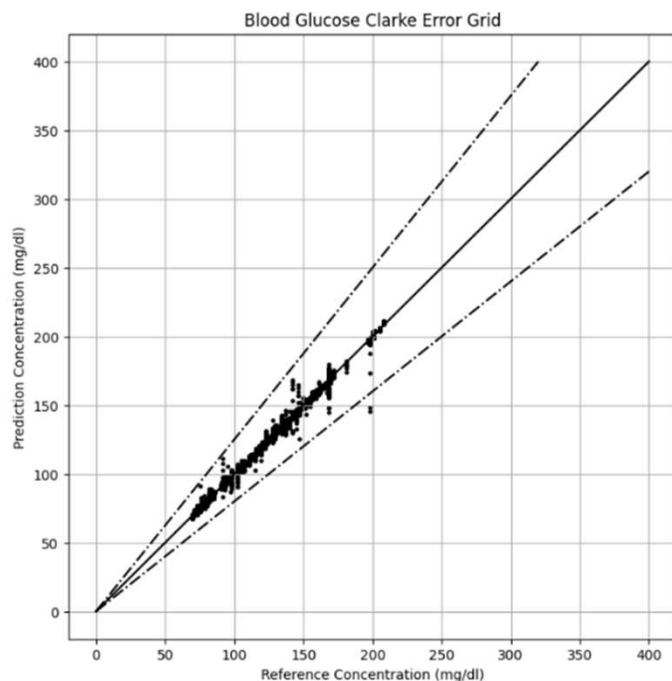
2. Clarke Error Grid (혈당 모니터링 장치의 정확성 평가 도구)

3. 별도의 검증 데이터셋을 사용하여 저장된 학습 결과와 실제 값을 비교 분석함으로써 모델의 정확도를 평가

모델 결과 1

기본 MLP 모델 평가 결과

학습 결과는 좋았지만
이상치 발견으로 과대적합 의심
=> 검증 결과 매우 안 좋음



Train Results:
MSE: 0.1168, MAE: 0.1777, RMSE: 0.3418, R^2 : 0.9996

Validation Results:
MSE: 0.1425, MAE: 0.1831, RMSE: 0.3775, R^2 : 0.9995

```
=====testy_predict=====
===== evaluate result real =====
=====Training Result=====
MAE: 51.66757005691528
MSE: 3329.0114982360997
RMSE: 57.69758658935484
R2 score: -31.956604783478346
=====Training Result=====
MAE: 41.91641428222657
MSE: 1756.9857862792476
RMSE: 41.91641428222657
R2 score: -11.956604783478346
```

성능 향상 실험 1

Robust Scaler 진행

Epoch 수 변경 (500 -> 300)

Batch 정규화

활성화 함수Relu 사용 -> 모델의 비선형 특성을 강화

```
PyTorch 라이브러리를 이용한 RobustScaler 사용  
scaler = RobustScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_val_scaled = scaler.transform(X_val)
```

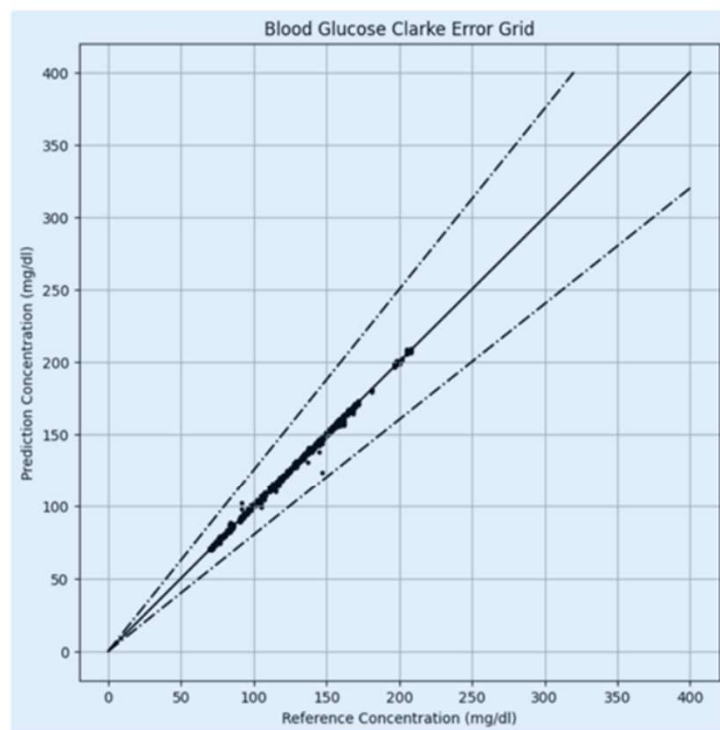
Train Results:

MSE: 0.5147, MAE: 0.2869, RMSE: 0.7174, R^2 : 0.9984

Validation Results:

MSE: 0.7148, MAE: 0.3017, RMSE: 0.8454, R^2 : 0.9977

파라미터 수정한 MLP 모델 결과



성능 향상 실험 2

은닉층 노드 수 조정

노드 수가 1,410 개 -> 모델 성능 가장 우수

입력층:1 은닉층:[64,128,256,512,1024] / 출력층:1 총 노드수 1984



입력층:1 은닉층:[64,128,256,512,1024,512,256,128,64] / 출력층:1 노드수 :2946



입력층:1 은닉층:[64,128,256,512,256,128,64] / 출력층:1 노드수 1410



입력층:1 은닉층:[64,128,300,512,256,128,64] / 출력층:1 노드수 1454

```
# TensorFlow를 모델 정의
def build_model(input_shape):
    model = Sequential()
    model.add(Dense(64, input_shape=input_shape))
    model.add(BatchNormalization()) # 배치 정규화 추가
    model.add(Activation('relu'))

    model.add(Dense(128))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(256))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(512))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(1024))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(256))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(128))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(64))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(1))
    return model

model = build_model(X_train_scaled.shape[1])
```

```
# TensorFlow를 모델 정의
def build_model(input_shape):
    model = Sequential()
    model.add(Dense(64, input_shape=input_shape))
    model.add(BatchNormalization()) # 배치 정규화 추가
    model.add(Activation('relu'))

    model.add(Dense(128))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(256))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(512))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(1024))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(256))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(128))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

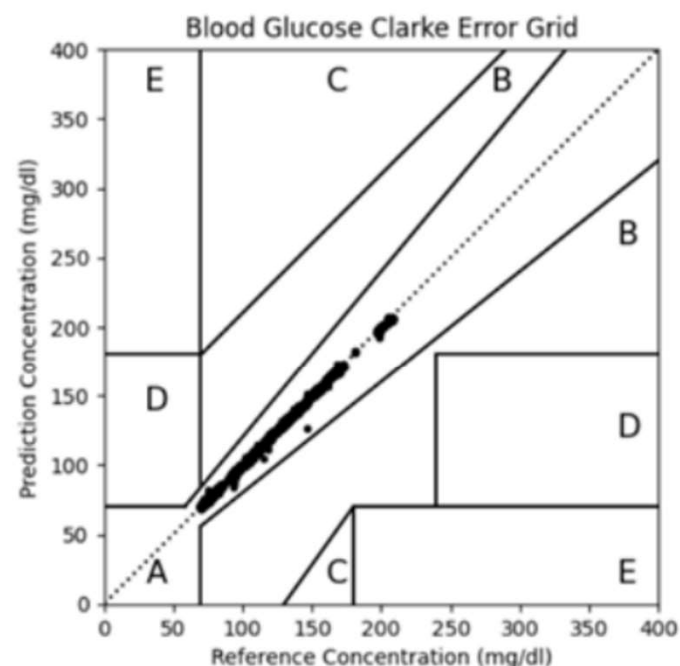
    model.add(Dense(64))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Dense(1))
    return model

model = build_model(X_train_scaled.shape[1])
```

최종 결과

TensorFlow MLP 모델 결과



Train Results:

MSE: 0.2710, MAE: 0.2786, RMSE: 0.5206, R^2 : 0.9991

Validation Results:

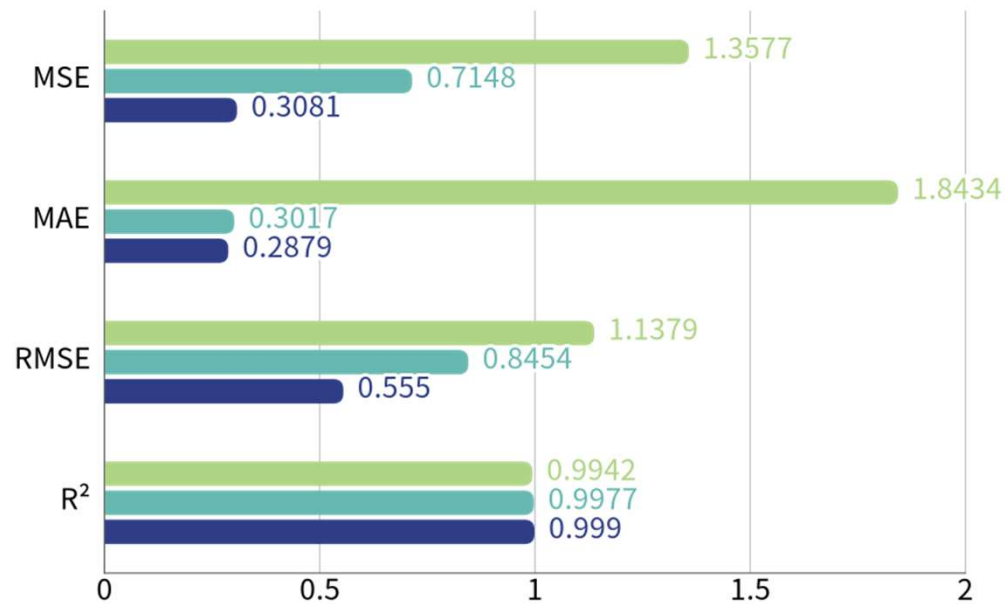
MSE: 0.3081, MAE: 0.2879, RMSE: 0.5550, R^2 : 0.9990

```
27  
28 print("혈당 실측 : 145 예측 : "+ str(predict_glucos))  
29  
30 print(sdhnlst100[0], rmssdlst100[0], pnn50lst100[0],
```

혈당 실측 : 145 예측 : 151

10.697 7.974 1e-08 8.795887 24.647115 57.291515 0.430205

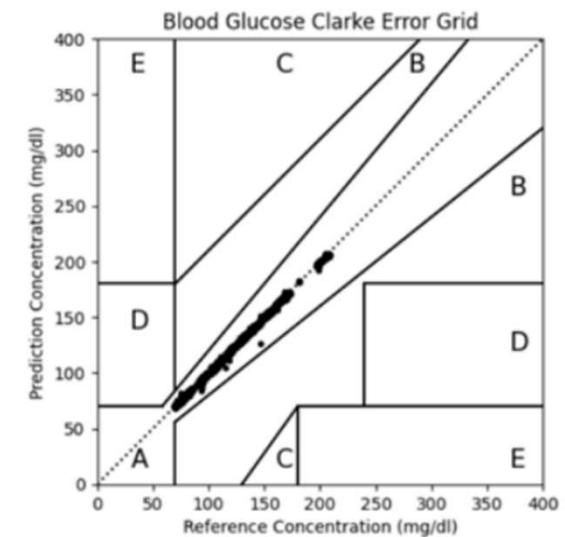
모델 비교



■ StandardScalerndard ■ Pytorch RobustScaler
■ TensorFlow RobustScaler

최종 채택

TensorFlow MLP 모델 결과



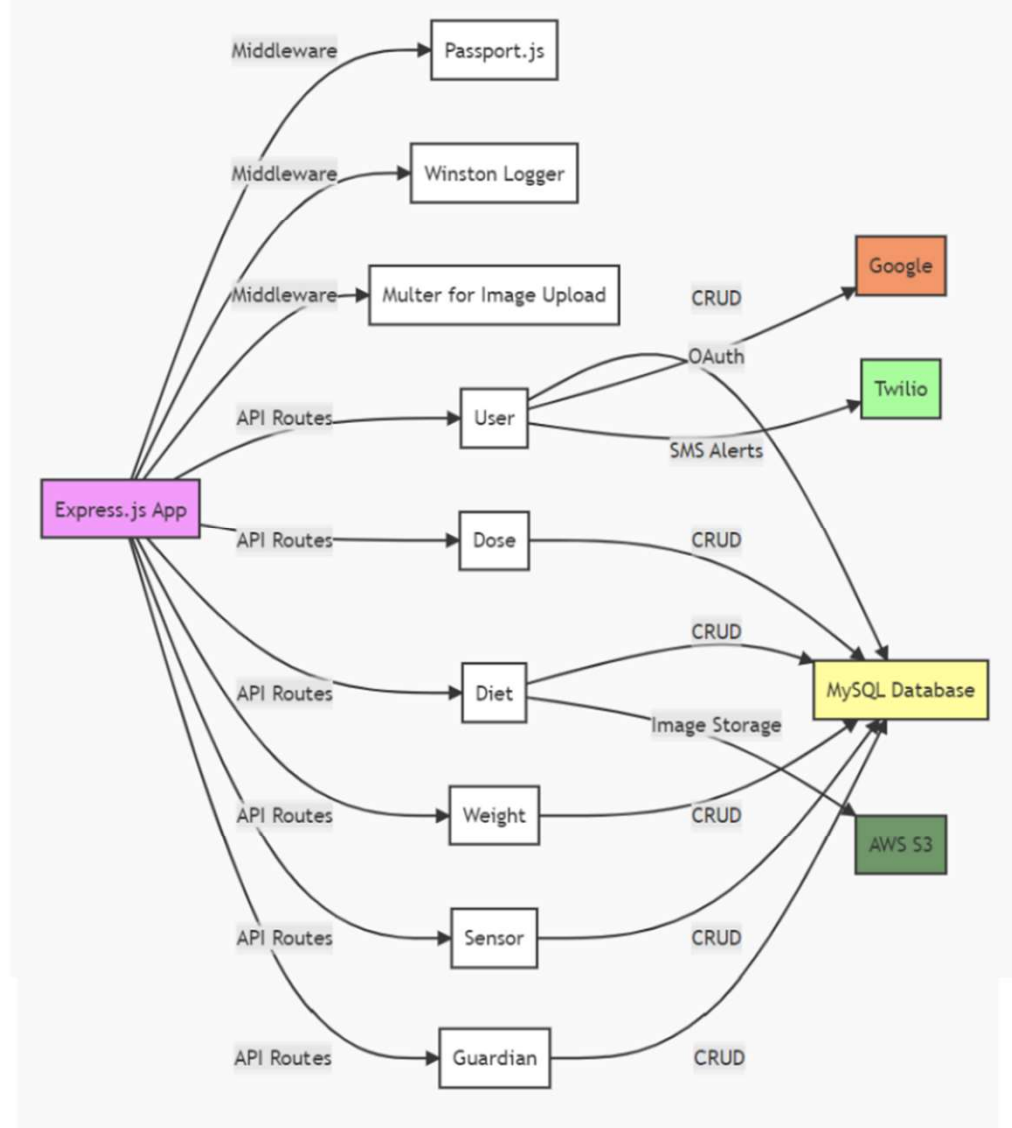
혈당 실측 : 145 예측 : 151

10.697 7.974 1e-08 8.795887 24.647115

서버 아키텍처 다이어그램

Node.js와 Express.js로 구축

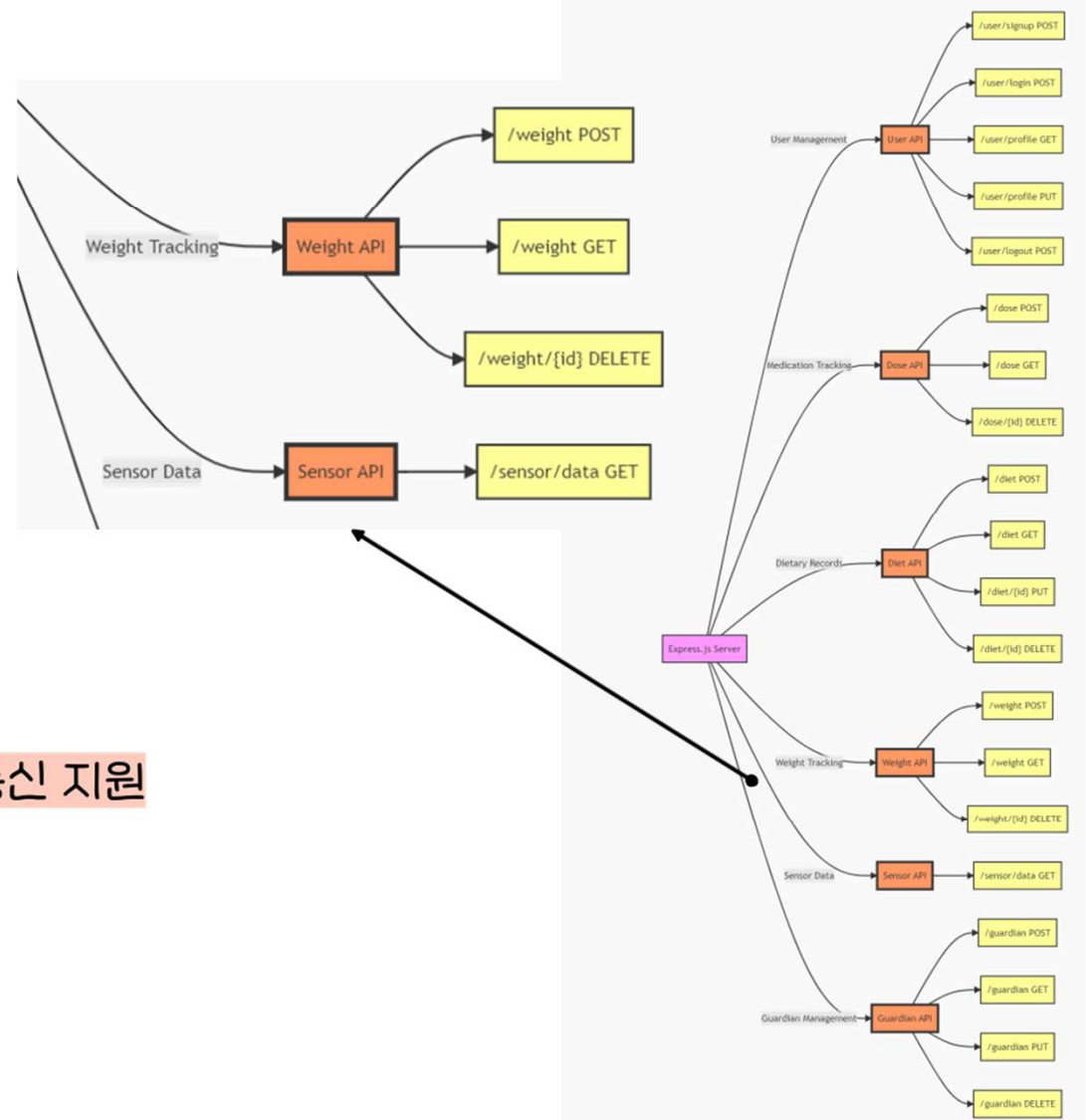
AWS EC2 에 서버 배포



API 엔드포인트 리스트

독립적인 API 엔드포인트로 모듈화
-> 시스템의 확장성과 유지보수가 용이

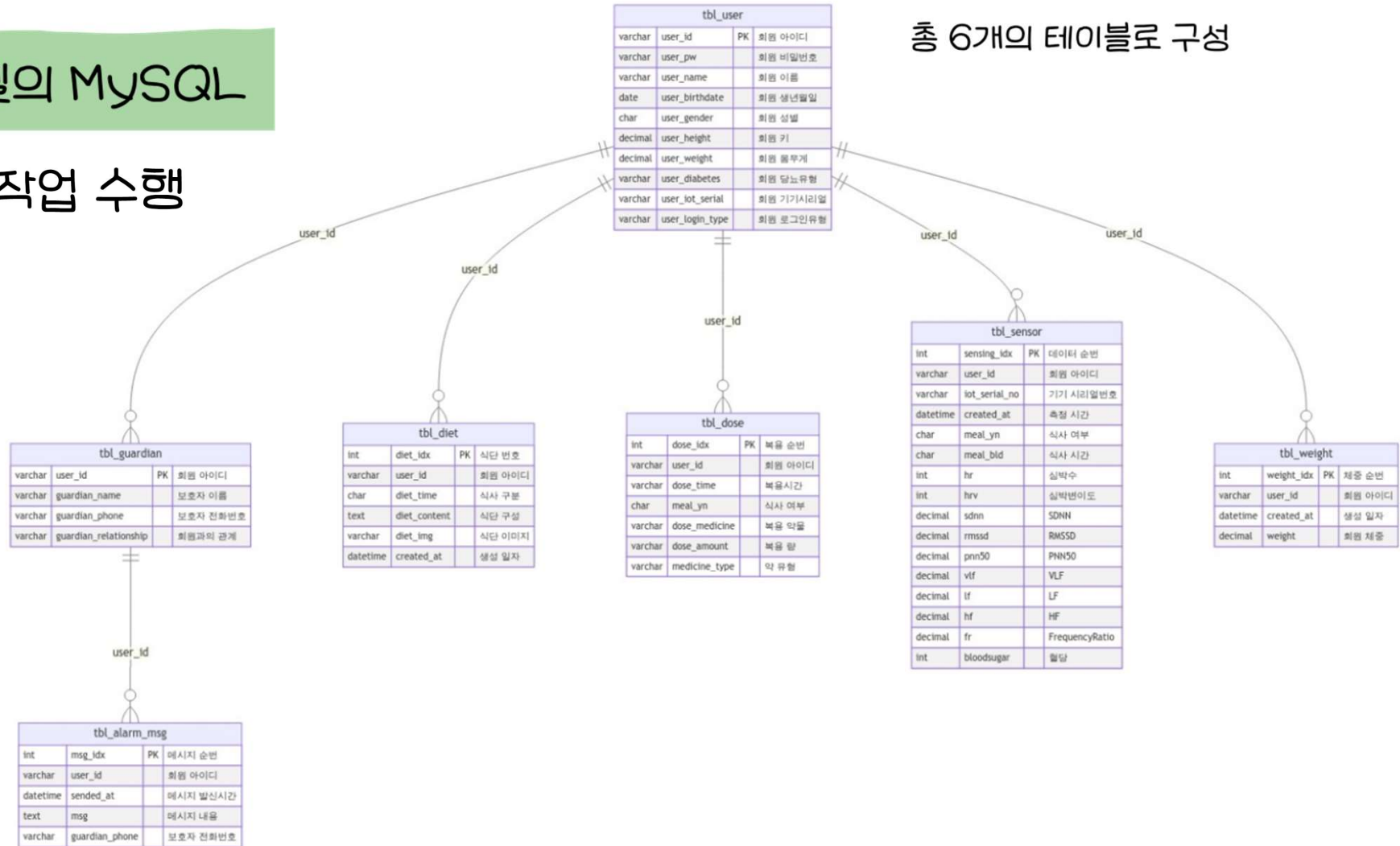
RESTful API 아키텍처 스타일 채택
-> 클라이언트 APP과 서버 간의 원활한 통신 지원



관계형 모델의 MySQL

CRUD 작업 수행

총 6개의 테이블로 구성



보안 및 암호화

비밀번호 단방향 해시 알고리즘인 SHA-256을 사용해 암호화

센서 데이터 AES 대칭키 암호화 알고리즘을 사용하여 암호화, 복호화

```
• INSERT INTO tbl_sensor(user_id, iot_serial_no, create
VALUES ('user_id_value', 'iot_serial_no_value', NOW()
AES_ENCRYPT('hr_value', 'cellu'),
AES_ENCRYPT('hrv_value', 'cellu'),
AES_ENCRYPT('sdnn_value', 'cellu'),
AES_ENCRYPT('rmssd_value', 'cellu'),
AES_ENCRYPT('pnn50_value', 'cellu'),
AES_ENCRYPT('vlf_value', 'cellu'),
AES_ENCRYPT('lf_value', 'cellu'),
AES_ENCRYPT('hf_value', 'cellu'),
AES_ENCRYPT('fr_value', 'cellu'),
AES_ENCRYPT('bloodsugar_value', 'cellu'));

• SELECT user_id, iot_serial_no, created_at, meal_yn, meal_bld,
CAST(AES_DECRYPT(hr, 'cellu') AS CHAR) AS hr,
CAST(AES_DECRYPT(hrv, 'cellu') AS CHAR) AS hrv,
CAST(AES_DECRYPT(sdnn, 'cellu') AS CHAR) AS sdnn,
CAST(AES_DECRYPT(rmssd, 'cellu') AS CHAR) AS rmssd,
CAST(AES_DECRYPT(pnn50, 'cellu') AS CHAR) AS pnn50,
CAST(AES_DECRYPT(vlf, 'cellu') AS CHAR) AS vlf,
CAST(AES_DECRYPT(lf, 'cellu') AS CHAR) AS lf,
CAST(AES_DECRYPT(hf, 'cellu') AS CHAR) AS hf,
CAST(AES_DECRYPT(fr, 'cellu') AS CHAR) AS fr,
CAST(AES_DECRYPT(bloodsugar, 'cellu') AS CHAR) AS bloodsugar
FROM tbl_sensor;
```

토큰 기반 인증 시스템 JWT(JSON Web Token)활용하여 사용자 세션 정보를 클라이언트 측에서 관리
로그아웃은 클라이언트 측에서 토큰을 삭제함으로써 이루어짐

서버 최적화

JWT 인증 강화

서버의 안정성과 성능 ↑

PM2 프로세스 관리

서버 운영의 안정성 ↑

쿼리 최적화

데이터 처리시간 ↓

비동기 프로그래밍 방식적용

서버 동시 처리 능력 ↑

시연 영상



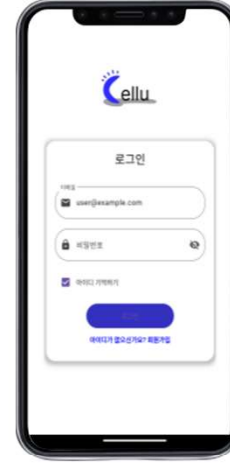
기기 전원을 켜 후 측정을 시작합니다



APP 소개

로딩 화면

로고 화면

로그인

ID : 이메일
PW : 비밀번호
비밀번호 표시, 숨기기 기능
아이디 기억하기

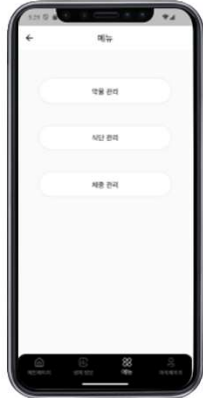
회원가입

ID : 사용자 이메일
PW : 비밀번호
비밀번호 확인
비밀번호 표시, 숨기기 기능

메인

년, 월, 일 날짜 선택
좌우 스크롤 기능
혈당 데이터 상세 보기
평균 혈당 수치
약물 복용 카드

메뉴 페이지



약물 작성 Pg 이동
식단 관리 Pg 이동
체중 관리 Pg 이동

식단 관리



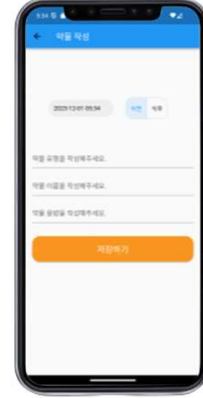
날짜 선택 기능
업로드한 식단 보기
식단 작성하기
사진, 글 업로드 기능

체중 관리



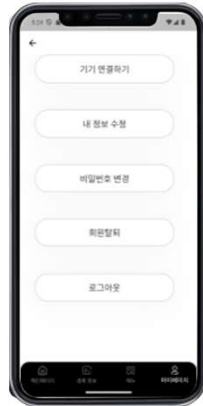
표준, 현재 체중 차이
체중 조절 필요 유무
운동 주의사항 정보

약물 작성



약물 복용 시간
식전 or 식후
약물 유형 작성
약물 이름 작성
약물 용량 작성

마이페이지



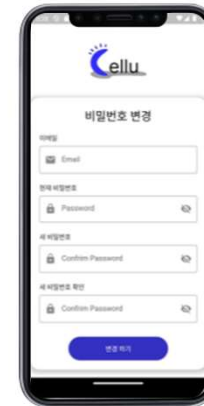
기기 연결 Pg 이동
내 정보 수정 Pg 이동
비밀번호 변경 Pg 이동
회원 탈퇴
로그아웃

내 정보수정



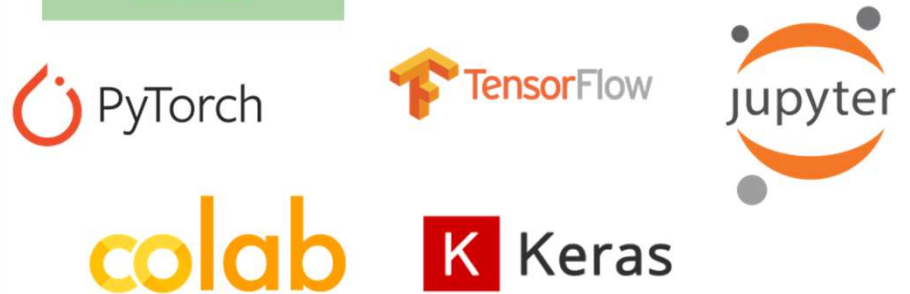
생년 월일 입력
성별 입력
키 입력
체중 입력
당뇨 유형 선택
보호자 추가

비밀번호 변경



아이디 (이메일) 입력
현재 비밀번호 입력
새로운 비밀번호 입력
확인용 비밀번호 입력
비밀번호 표시, 숨기기 기능
비밀번호 일치 확인 후 변경

모델링



Back-End



Front-End



DataBase



협업 도구





3. 활용방안 및 기대 효과

기대 효과

저렴한 유지 비용



편리한 혈당 측정



소아 당뇨 환자의 부담 ↓

건강 모니터링의 효율성 ↑

활용 방안

질병 예방, 관리에 중요한 역할

소셜 로그인 및 개인화된 식단 추천 도입

Q & A

감사합니다

팀원 소개



팀장 | 박승주
PM, 기획, UI/UX담당자

프로젝트 기획
산출문서 작성 및 취합
UX/UI 설계
메인 페이지 구현 (Flutter)
생체 정보 페이지 구현 (Flutter)
측정 페이지 구현 (Flutter)
측정 에러 페이지 구현 (Flutter)
기기 연결 페이지 구현 (Flutter)



팀원 | 이규해
Back-End 담당자

Node.js 기반 백엔드 서버 구현
AWS EC2 서버 구동(백엔드, 예측 모델)
로딩 페이지 구현 (Flutter)
로그인 페이지 구현 (Flutter)
회원 가입 페이지 구현 (Flutter)



팀원 | 장서우
모델링 담당자

Pytorch, Tensorflow 사용
Z-Score scaler 기반
MLP 모델 전처리, 설계, 검증
내 정보 수정 (Flutter)
비밀번호 변경 (Flutter)



팀원 | 김슬기
모델링 담당자

Pytorch, Tensorflow 사용
Robust scaler 기반
MLP 모델 전처리, 설계, 검증
메뉴 페이지 (Flutter)
하단 네비게이션 (Flutter)
마이페이지 구현 (Flutter)
약물 작성 페이지 구현 (Flutter)



팀원 | 박성현
데이터 베이스 담당자

MySQL 사용
AES 암호화/복호화
체중관리 페이지 구현 (Flutter)
식단 관리 페이지 구현 (Flutter)
식단 작성 페이지 구현 (Flutter)